

Musical taste prediction using demographic traits, personality traits and health habits

Vanja Vujović, 56m/25

Introduction

Machine learning represents an algorithmic approach to finding patterns in a dataset. It is the process from data normalization, generalization and transformation, to creating a valid model which can answer questions regarding our dataset. Possible learning patterns it can tackle are classification, number prediction, association and clustering. In this paper, we will create a model based on a personality questionnaire that aims to predict the musical genres a person is most likely to enjoy. We will create our models using **python** with the **scikit-learn** library.

Notion

- *Training set* - part of the dataset used to train a model.
- *Test set* - part of the dataset used to evaluate a model
- N - Total number of instances in the training set.
- A_i - The i -th attribute (or feature) within the dataset.
- E - A specific data instance, represented as a vector of observed attribute values.
- $a(i)$ - i -th instance in the training set, represented as a vector of observed attribute values.
- E_i - The realized value of the i -th attribute, A_i , for a given instance E .
- K_i - Number of distinct values of attribute A_i
- C_j - The j -th discrete label of the target class.
- N_j - Total number of training instances belonging to class C_j
- $N_j(E_i)$ - Count of training instances belonging to class C_j with feature A_i has value E_i

Dataset

We have a dataset with 1010 instances representing answers to a young people survey, with people ranging from 15 to 30 years. It contains 150 columns, from which we will predict 19 class attributes based on 70 attributes. Class attributes represent likeability of certain musical genres as a rating from 1-5 (strongly disagree to strongly agree). 54 attributes are represented the same way, 5 are numerical, and the remaining 11 are categorical, although most of these can be analyzed as ordinal or binary.

Class attributes preprocessing

Whether a person likes a given genre is a binary classification problem, with two possible outcomes: yes (positive class) or no (negative class). We changed our ratings to this format by mapping ratings of 4 and 5 to the positive class and ratings of 1 and 2 to the negative class. Ratings of 3, representing a neutral preference, as well as missing ratings, were treated as missing values (NaN).

Attributes preprocessing

Most of the models we will be using require our attributes to be non-missing and numerical. For each attribute, we populate missing values with the most frequent value. To make our data numerical, we have to change categorical values to numerical. To do this, we divided the 11 categorical attributes into the following groups.

1) Binary

Gender - male (0) / female (1)

Left or right handed - Right handed (1), Left handed (0)

Village or town - Village (1), City (0).

House or block of flats - House/Bungalow (1), Block of flats (0)

2) Ordinal

Internet usage - no time at all (0), less than an hour a day (1), few hours a day (2), most of the day (3)

Education - currently a primary school pupil (0), primary school (1), secondary school (2), college/bachelor degree (3), masters degree (4), doctorate degree (5)

Alcohol - never (0), social drinker (1), drink a lot (2)

Smoking - never smoked (0), tried smoking (1), former smoker (2), current smoker (3)

3) Redundant

Only child - as we have a column *Number of siblings*, this is redundant. However, before removing it we correct *Number of siblings* by writing 0 everywhere where *Only child* is true.

4) Categorical

Lying - never, sometimes, only to avoid hurting someone, everytime it suits me

Punctuality - I am often early, I am always on time, I am often running late

For categorical features *Lying* and *Punctuality*, each distinct value E_i is converted into a separate binary column. The value is set to **1** if the instance has E_i as the value of the original feature A_i , and **0** otherwise. This technique is called **One-Hot Encoding**.

For some machine learning models, one of the binary columns can be removed. This is because its value can be inferred from the remaining columns: if all the remaining binary values are **0**, the omitted category must be **1**, otherwise it must be **0**.

Data split

As we want to properly evaluate models, we need to split the data into training and test sets. The training set will be used to train and tune our models, while the test set will be used to evaluate our final model. We will use an **80/20** split ratio. We also want both positive and negative classes from each class attribute to be split between sets in this ratio, to ensure training and test data are good representations of the whole dataset.

When we have only 1 class label, this is simply done by dividing instances into **80/20** splits for each class label separately, and then uniting the splits to get a full **80/20** split (**stratification**). The goal is the same when we have multiple class attributes, have each class label for each class attribute divided closely to **80/20** ratio in our splits. The problem with standard splitting methods is that putting an instance in one of the splits affects the ratios of all class attributes simultaneously (Szymański & Kajdanowicz, 2017), so trying to balance "Rock" classes might disbalance "Pop" classes. To attack this problem, we will use **MultilabelStratifiedShuffleSplit** implemented inside the **iterative-stratification** library. It first finds a class attribute with the lowest number of non-distributed positive instances, and then arranges those with ratio **80/20** (this takes into account already distributed positive instances of the given class attribute). After this, it recalculates label counts for non-distributed instances and label ratios for distributed ones, and repeats the process until all instances are separated.

Model selection with multiple class attributes

We have multiple class attributes, so we are dealing with a multi-labeling problem. This means that for each person, we want to predict possibly multiple likeable genres, which differs from normal classification problems where we predict only 1 class per instance. We will do this by using a technique called *binary relevance*. We create 1 separate classification model for each class attribute which predicts the probability that a given person likes a given genre. After this, we predict a set of genres which the person will like by choosing top n answers sorted by their perspective probabilities, excluding ones we filter by probability thresholds which we will calculate for each genre separately. We will pick the best models from a pool of models which can perform well for the given problem. We will now go through the metrics and the models we will be using.

Classification metrics

When a classification model is trained and predicts a single instance, we can categorize our prediction in following manner:

True Positive (TP) - we predicted the positive class correctly

False Positive (FP) - we predicted the positive class incorrectly

True Negative (TN) - we predicted the negative class correctly

False Negative (FN) - we predicted the negative class incorrectly

Now, we can define the metrics we will be using to evaluate our classification models:

Precision: how precise our positive class predictions are $\frac{TP}{TP + FP}$

Recall: how much positive class instances we predicted correctly $\frac{TP}{TP + FN}$

F1: harmonic mean of *precision* and *recall* $\frac{2 * precision * recall}{precision + recall}$

log loss: how far our probability predictions are from the actual values on average.

$$Log\ loss = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

roc auc: how well our model ranks probabilities of positive instances above the negative ones.

X1 - predicted probability for random positive instance

X2 - predicted probability for random negative instance

$$Roc\ auc = P[X1 > X2] + \frac{1}{2}P[X1 = X2]$$

Multi labeling metrics

When we have multiple binary class attributes (as our final model will have) and we predict a set of **n** positive classes (**S1**) for a single instance, we need to evaluate this in comparison to the set of actual positive classes (**S2**). We will define our metrics based on set operations on these sets.

Precision@n: how precise our positive class predictions are for given instance $\frac{|S1 \cap S2|}{n}$

Recall@n: how much positive classes we covered for given instance $\frac{|S1 \cap S2|}{|S2|}$

F1@n: harmonic mean of *precision@n* and *recall@n* $\frac{2 * precision@n * recall@n}{precision@n + recall@n}$

Mean precision@n: *Precision@n* average on the test set

Mean recall@n: *Recall@n* average on the test set

Mean f1@n: *f1@n* average on the test set

Decision tree

Decision tree is one of the most interpretable models for classification. Firstly, a split of instances is created based on some constraint. For example, we split the instances on attribute *age* in sets "*age < 18*" and "*age >= 18*" or on attribute *interests* in sets "*interests = football*" and "*interests != football*". To choose the best split, an optimization criteria for minimizing the impurity of the split (we want both splits to have a lot of instances of the same class, so we can confidently predict those classes) is used, such as *entropy* or *gini index*. This is then recursively repeated for each set, until all of the instances are of the same class (0% impurity is reached), or other threshold criteria holds (maximum tree depth or minimum number of instances per leaf node). To evaluate a new instance, we just move it down the tree by following conditions which hold for it. By looking at rule paths from top of the tree to the bottom, we can easily interpret how the instances are divided.

eXtreme Gradient Boosting

eXtreme Gradient Boosting (XGBoost) is a continuation technique on decision trees. The idea is that, instead of creating one tree which tries to fully correctly categorize the training set, this algorithm creates multiple shallow trees. After the first tree is created, the second tree is not trained on the class labels, but rather on the difference (residual) between class labels and predictions of the first tree, after which the third tree is trained on difference between class labels and sum of predictions of first 2 trees and so on. By summing predicted values of each tree for our instance, we get our final classification.

K nearest neighbors

K nearest neighbors (**KNN**) is one of the simplest classification models. It represents each instance as a numeric vector in vector space, and defines a distance function between 2 vectors. Finally, it classifies a given instance by taking the most frequent class of k nearest instances from the training set based on the given distance function. The most widely used distance function is *Euclidean* for numerical, and *Hamming* for categorical attributes (which is just 0 if categories are the same and 1 otherwise). This technique usually requires standardized or normalized numerical values, so that attribute scales don't influence the distance function.

Logistic regression

Linear regression is a model which creates a linear combination (with bias coefficient) of attributes to predict a numerical class attribute. After the coefficients are calculated, we get a numeric output which represents the class attribute by inputting instance attribute values inside the linear combination.

$$y = b + w_1x_1 + \dots + w_nx_n$$

To calculate the best coefficients, method of *minimum squared error* is used, which finds a hyperplane that has minimum sum of squared distances from all of the instances in the training set.

Logistic regression is an extension to this algorithm which allows us to use it for classification. As linear regression can give output ranging from $(-\infty, \infty)$, we want a function to reduce this interval onto $[0, 1]$ so we can interpret this as the probability that an instance belongs to the positive class. A well known function which does this is **sigmoid function**.

$$\sigma(y) = \frac{1}{1 + e^{-y}}$$

As this is not a linear equation anymore, we iteratively minimize the cost function to calculate these coefficients. **Log loss** is used for cost function, as its derivative in respect to coefficients is easy to calculate, which is needed because minimizing techniques such as *Gradient Descent* and *Iteratively Reweighted Least Squares (IRLS)* rely on iterative calculation of partial derivatives.

Naive bayes

Naive Bayes is a probabilistic algorithm used for classification. For each possible class value, it calculates the probability that the given instance belongs to it, and returns the class with the highest probability. To do this, it assumes that distributions of the attributes are independent, and then utilizes the Bayes formula for conditional probability.

$$P(Cj|E) = \frac{P(E|Cj) * P(Cj)}{P(E)} = \frac{P(E|Cj) * P(Cj)}{\sum_{i=1}^n P(E|Ci) * P(Ci)}$$

Now, to calculate $P(E|Cj)$, we can use the independence assumption:

$$P(E|Cj) = \prod_{i=1}^n P(Ei|Cj) \qquad P(Cj) = \frac{N_j}{N}$$

To find $P(Ei|Cj)$, several approaches are possible resulting in different models, from which we will use the following ones:

1) *Categorical Naive Bayes*

Here, we assume every attribute is categorical and discrete, and calculate our probabilities using counts. We add a tunable constant α to each count, so we don't get 0 probability if some category was not seen during the training.

$$P(E_i|C_j) = \frac{N_j(E_i) + \alpha}{N_j + \alpha K_i}$$

2) *Bernoulli Naive Bayes* - this is the same as *Categorical*, but with each attribute being binary (so only 2 categories). We binarize the attributes based on their midpoint for this algorithm.

3) *Gaussian Naive Bayes* - for a fixed attribute A_i and all instances of class C_j , the mean (μ_{ij}) and standard deviation (σ_{ij}) are calculated, after which we approximate the distribution as *Gaussian (normal)*.

$$P(E_i|C_j) = \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} e^{-\frac{(x-\mu_{ij})^2}{2\sigma_{ij}^2}}$$

Note: for *Categorical* and *Bernoulli* versions, we have to remove continuous data (*age, weight, height, and number of siblings*) as they work only with discrete data.

Support Vector Classification

One of the problems with linear regression is that it can only look for linear dependencies between attributes and class. If we for example wanted a quadratic dependency (instead of radius, we need circle area), we would need to add this column to the database before performing linear regression. To solve this problem, *Support Vector Classification (SVC)* was created, which can look for polynomial, trigonometric, exponential dependencies etc. without the need of explicit column creation, which reduces the computation cost exponentially. It represents the previous linear equation as an equation with dot products of the current instance with *support vectors*, which represent vectors that define the optimal hyperplane.

$$y = b + \sum \alpha_i y_i a(i) \cdot a$$

Here, y is predicted value (negative means class **0**, positive **1**), α_i is a Lagrange multiplier which is learned and **0** for non support vectors, y_i is class value of the i -th instance in the training set, and a is current instance. We can now abstract the dot product $a(i) \cdot a$ with kernel function

$K(a(i), a)$, and by using different implementations of this function obtain the dependencies we want, without any new columns being created.

Rule based algorithms

Rule based algorithms divide instances based on rule sets or lists. Each rule is composed of conditions (such as "*age < 18*" or "*time spent = little*"), and these are combined using logical operators (usually with $\wedge$ or $\vee$ but not necessarily). Usual algorithms use an approach called **separate and conquer**: they build rules 1 by 1; after a rule is built they *separate* the instances which are covered by the rule, after which they repeat the process on the remaining instances until all instances are covered (or stopping criteria are met). To build a single rule, they build it by greedily choosing the best condition at each step (usually by impurity), until all rule instances have the same class attribute or other criteria are reached (minimum number of instances per rule or maximum number of conditions in a rule). This approach can have overlapping rules or non-classifiable instances. Fortunately, both of those problems are easily solved when we work with binary class attributes: we create rules only for the positive class, and if an instance is not covered, it is classified as negative. We use 2 models which implement this approach: *RIPPER* follows the logic from above, but it learns rules for one class label at a time, going from least to most frequent. When it generates all of the rules, it tries to modify the rules a bit to possibly get better results. Also, our implementation only uses equality for its conditions. *GreedyRuleList* creates a rule by following the *DecisionTree* approach. It creates a split condition greedily by minimizing impurity, but instead of splitting data into 2 sets, it only looks and goes recursively into set with smaller impurity of positive class, thus not creating a tree but a single condition chain which is our rule. It only creates one chain rule.

0R

0R (DummyClassifier) is a simple classification model which always predicts the majority class with probability equal to its proportion in the training set. This classifier is mostly used as the borderline model which our real model has to beat to have any meaningful purpose.

Probability calibration

Since our models are trained separately for each genre, their probability predictions may be uncalibrated (*Niculescu-Mizil & Caruana, 2005*) and therefore not directly comparable across genres. To address this, we wrap each model in *CalibratedClassifierCV*, which learns a mapping from the model's raw outputs to calibrated probabilities. After calibration, a predicted probability of, for example, *80%* means that, among a large group of instances assigned probabilities around *80%*, approximately *80%* are expected to belong to the positive class. By calibrating the models for each genre this way, their probability predictions become more suitable for comparison.

Single genre results

To choose and tune the best model for each genre, we need to compare our models based on the training data alone, as interfering the test set with our decisions will make the model biased. We will evaluate each model using *StratifiedKfold*. We split our training data into 5 folds, train our model on 4 of them and evaluate on the 5th. This is repeated for each possible 5th fold and the average of **log loss** on these is taken. For each genre, we use this metric to choose the best model. We exclude instances with NaN value for the current genre from training.

It is worth noting that for robustness purposes, we also tried using **roc auc**, **precision** and **recall** as our metric, but all of these gave similar final results, thus emphasizing that our results are not biased on the metric choice. In the table below, for each genre we represent the best model from the model selection process, with its tuned parameters, **log loss** and **roc auc** score.

GENRE	MODEL	PARAMETERS	LOG LOSS	ROC AUC
Dance	SVC	gamma: scale, kernel: rbf	0.6552	0.6576
Folk	GaussianNB		0.4592	0.6558
Country	SVC	gamma: auto, kernel: poly	0.4079	0.6224
Classical music	SVC	gamma: auto, kernel: rbf	0.6528	0.6673
Musical	BernoulliNB	alpha: 2.0, binarize: 0.3	0.6353	0.6578
Pop	XGBClassifier	learning_rate: 0.1, max_depth: 4, n_estimators: 100	0.58	0.6612
Rock	SVC	gamma: scale, kernel: rbf	0.4811	0.646
Metal or Hardrock	SVC	gamma: scale, kernel: rbf	0.5684	0.6532
Punk	SVC	gamma: scale, kernel: rbf	0.6037	0.6029
Hiphop, Rap	SVC	gamma: auto, kernel: rbf	0.637	0.6927
Reggae, Ska	SVC	gamma: scale, kernel: rbf	0.6616	0.5975
Swing, Jazz	SVC	gamma: scale, kernel: rbf	0.6507	0.6385
Rock n roll	BernoulliNB	alpha: 0.5, binarize: 0.3	0.6662	0.6275
Alternative	XGBClassifier	learning_rate: 0.2, max_depth: 5, n_estimators: 200	0.6588	0.6419
Latino	SVC	gamma: scale, kernel: rbf	0.605	0.7193
Techno, Trance	SVC	gamma: auto, kernel: rbf	0.5753	0.5969
Opera	SVC	gamma: scale, kernel: poly	0.4695	0.5888

Now, we compare **log loss** between our best models and the *DummyClassifier*.

GENRE	MODEL	LOG LOSS	OR LOG LOSS	REL_IMPROV	POSITIVE CLASS
Dance	SVC	0.6552	0.6878	4.74%	55.17%
Folk	GaussianNB	0.4592	0.4827	4.87%	18.76%
Country	SVC	0.4079	0.4153	1.78%	14.57%
Classical music	SVC	0.6528	0.6915	5.60%	47.10%
Musical	BernoulliNB	0.6353	0.6686	4.99%	38.97%
Pop	XGBClassifier	0.58	0.6029	3.79%	70.92%
Rock	SVC	0.4811	0.5004	3.85%	80.00%
Metal or Hardrock	SVC	0.5684	0.5975	4.87%	28.49%
Punk	SVC	0.6037	0.6157	1.95%	30.58%
Hiphop, Rap	SVC	0.637	0.6918	7.93%	47.44%
Reggae, Ska	SVC	0.6616	0.6725	1.62%	39.86%
Swing, Jazz	SVC	0.6507	0.6788	4.13%	41.54%
Rock n roll	BernoulliNB	0.6662	0.6863	2.93%	55.83%
Alternative	XGBClassifier	0.6588	0.6831	3.56%	42.93%
Latino	SVC	0.605	0.6797	11.00%	41.82%
Techno, Trance	SVC	0.5753	0.5862	1.86%	27.30%
Opera	SVC	0.4695	0.4744	1.02%	18.20%

As we can see, all genres have a **log loss** improvement over the **OR** baseline. However, genres like *Techno*, *Trance*, *Country* and *Opera* show very small improvements, partially because **log loss** heavily penalizes overconfident incorrect predictions. In contrast, the **roc auc** metric shows a much clearer performance improvement, thus emphasizing that looking at only 1 metric may not be enough to make proper evaluation. Another possible reason for smaller improvements for some genres can be the shortage of positive class instances.

Probability thresholds

To select the top recommended genres for each person, we combine our best models for separate genres. Since each model predicts a probability that a user likes that specific genre, we find model-specific probability thresholds to determine positive recommendations. We do this on the training data, by reusing the same *StratifiedKfold* strategy mentioned above. We find these thresholds by testing probabilities from 5% to 50% in 5% increments, selecting the value that maximizes average *F1* score across folds for each model, as we want to maximize both *recall* and *precision*. In the table below we give the calculated thresholds for each genre.

Dance	Folk	Country	Classical music	Musical	Pop	Rock	Metal or Hardrock	Punk
40%	20%	15%	40%	30%	45%	50%	25%	25%

Hiphop, Rap	Reggae, Ska	Swing, Jazz	Rock n roll	Alternative	Latino	Techno, Trance	Opera
40%	30%	35%	40%	35%	35%	25%	15%

Final model

We predict the top recommended genres for a given user in the following manner:

- 1) We predict the probability for each of the genres using our trained classification models.
- 2) We filter all genres whose probability is below our calculated thresholds.
- 3) We sort the rest of the genres by the predicted probability, and pick the best *n*.

To estimate our model performance, we calculated the multi-label metrics mentioned above.

MODEL	PRECISION@3	RECALL@3	F1@3	PRECISION@5	RECALL@5	F1@5
Final Model	56.06%	36.92%	40.85%	50.81%	53.18%	47.93%
Dummy Model	54.21%	33.28%	38.21%	47.47%	50.51%	44.92%
Random model	29.97%	13.61%	18.11%	38.99%	39.75%	36.50%
DecisionTree	53.70%	35.02%	38.87%	47.47%	49.49%	44.65%

Dummy model represents always picking top *n* most frequent classes, *Random model* predicts *n* classes at random (this model is only for reference), and last is our *DecisionTree* tuned model. As we can see, our model beats both the *Dummy model* and *DecisionTree* on every metric. However, it doesn't beat them by much, thus emphasizing that signal is weak between individual differences and specific genres.

Interpretable rules

Now that we have presented our final model, we also want to find and interpret some rules regarding our dataset. To do so, we will analyze the outputs of the *DecisionTree* and *Rule Based Models* to find explicit patterns between attributes and classes.

After running *DecisionTree* for each genre, we found 280 rules which provide fully pure sets. We present 10 of them with largest support (number of instances covered):

DISLIKES Opera: Charity $\leq 2.50 \wedge$ Reliability $\leq 3.50 \wedge$ Workaholism ≤ 3.50 (93/665 instances covered)
DISLIKES Metal or Hardrock: Female $> 0.50 \wedge$ Life struggles $> 1.50 \wedge$ Weight $\leq 75.50 \wedge$ Giving $> 2.50 \wedge$ Criminal damage $\leq 3.50 \wedge$ Education $> 0.50 \wedge$ Decision making $\leq 4.50 \wedge$ Waiting $\leq 3.50 \wedge$ Healthy eating ≤ 3.50 (74/674 instances covered)
DISLIKES Opera: Charity $\leq 2.50 \wedge$ Reliability $> 3.50 \wedge$ Age $\leq 28.50 \wedge$ Hypochondria $\leq 1.50 \wedge$ Funniness $\leq 4.50 \wedge$ Workaholism $\leq 4.50 \wedge$ Appearance and gestures $\leq 4.50 \wedge$ Weight ≤ 93.50 (70/665 instances covered)
DISLIKES Punk: Hypochondria $\leq 3.50 \wedge$ Workaholism $\leq 2.50 \wedge$ Village $\leq 0.50 \wedge$ Height $\leq 186.00 \wedge$ Getting up ≤ 4.50 (41/618 instances covered)
DISLIKES Opera: Charity $> 2.50 \wedge$ Questionnaires or polls $\leq 4.50 \wedge$ Assertiveness $\leq 4.50 \wedge$ Public speaking $> 2.50 \wedge$ Age $\leq 27.50 \wedge$ Thinking ahead $> 3.50 \wedge$ Getting up > 2.50 (40/665 instances covered)
LIKES Pop: Knowing the right people $> 2.50 \wedge$ Mood swings $> 2.50 \wedge$ Lying_never $\leq 0.50 \wedge$ Health $> 1.50 \wedge$ Age $\leq 22.50 \wedge$ Giving $> 2.50 \wedge$ Eating to survive $\leq 3.50 \wedge$ Healthy eating $> 2.50 \wedge$ Elections ≤ 2.50 (39/595 instances covered)
DISLIKES Swing, Jazz: Workaholism $\leq 2.50 \wedge$ Loneliness $\leq 3.50 \wedge$ Decision making $\leq 3.50 \wedge$ Changing the past $\leq 4.50 \wedge$ Weight $\leq 76.50 \wedge$ Charity ≤ 2.50 (39/573 instances covered)
DISLIKES Alternative: Elections $\leq 3.50 \wedge$ Achievements $\leq 3.50 \wedge$ Waiting $\leq 3.50 \wedge$ Hypochondria $\leq 3.50 \wedge$ Parents' advice $> 2.50 \wedge$ Alcohol $\leq 1.50 \wedge$ Getting angry $> 1.50 \wedge$ Health > 3.50 (38/601 instances covered)
LIKES Pop: Knowing the right people $> 2.50 \wedge$ Mood swings $> 2.50 \wedge$ Lying_never $\leq 0.50 \wedge$ Health $> 1.50 \wedge$ Age $> 22.50 \wedge$ Cheating in school > 2.50 (36/595 instances covered)
LIKES Dance: Energy levels $> 3.50 \wedge$ Appearance and gestures $> 4.50 \wedge$ Borrowed stuff $> 1.50 \wedge$ Compassion to animals $> 3.50 \wedge$ Mood swings > 1.50 (35/551 instances covered)

We now present the rule sets decided by the *RIPPER* algorithm for each genre:

GENRE	RULES
Dance	(Personality=4 $\wedge$ Happiness in life=4) $\vee$ (Socializing=4) $\vee$ (Fake=1) $\vee$ (Energy levels=5) $\vee$ (Mood swings=3 $\wedge$ Loneliness=2)
Folk	(God=5 $\wedge$ Questionnaires or polls=3)
Country	(Smoking=0 $\wedge$ Giving=5) $\vee$ (Giving=3 $\wedge$ Writing notes=3 $\wedge$ Reliability=4) $\vee$ (God=5 $\wedge$ Achievements=4 $\wedge$ Waiting=3)
Classical music	(Small - big dogs=3 $\wedge$ Eating to survive=1) $\vee$ (Socializing=1) $\vee$ (Keeping promises=5 $\wedge$ Alcohol=1) $\vee$ (Weight<52.0 $\wedge$ Prioritising workload=1)
Musical	(Female=1 $\wedge$ Smoking=0) $\vee$ (Female=1 $\wedge$ Keeping promises=4) $\vee$ (Assertiveness=4 $\wedge$ Questionnaires or polls=3) $\vee$ (Reliability=4 $\wedge$ Empathy=5) $\vee$ (Empathy=4 $\wedge$ Criminal damage=2)
Pop	(Unpopularity=4 $\wedge$ Happiness in life=4 $\wedge$ Punctuality_i am often early=0) $\vee$ (Knowing the right people=4 $\wedge$ Female=1 $\wedge$ Lying_sometimes=1) $\vee$ (Appearance and gestures=5)
Rock	(Dreams=3 $\wedge$ Workaholism=3) $\vee$ (Height175.0 - 178.0)
Metal or Hardrock	(Female=0 $\wedge$ Friends versus money=5) $\vee$ (Alcohol=2 $\wedge$ Mood swings=3) $\vee$ (Healthy eating=4 $\wedge$ Achievements=4) $\vee$ (Alcohol=0 $\wedge$ Lying_sometimes=0) $\vee$ (Workaholism=4 $\wedge$ Assertiveness=2) $\vee$ (Weight>84.0 $\wedge$ Empathy=5)
Punk	(Small - big dogs=4 $\wedge$ Elections=5 $\wedge$ Assertiveness=5)
Hiphop, Rap	(Cheating in school=5) $\vee$ (Empathy=3 $\wedge$ Small - big dogs=4) $\vee$ (Decision making=2 $\wedge$ Interests or hobbies=5) $\vee$ (Writing notes=4 $\wedge$ Keeping promises=5)
Reggae, Ska	(Alcohol=2 $\wedge$ Happiness in life=4)
Swing, Jazz	(Small - big dogs=3 $\wedge$ Friends versus money=5 $\wedge$ Daily events=2 $\wedge$ Keeping promises=4) $\vee$ (Health=3 $\wedge$ Alcohol=2 $\wedge$ Decision making=3)
Rock n roll	(Cheating in school=3 $\wedge$ Reliability=5) $\vee$ (Elections=5 $\wedge$ Weight56.0 - 60.0) $\vee$ (Punctuality_i am often early=0 $\wedge$ Hypochondria=4)
Alternative	(Elections=5 $\wedge$ Punctuality_i am always on time=1 $\wedge$ Judgment calls=5) $\vee$ (Responding to a serious letter=4 $\wedge$ Small - big dogs=3)
Latino	(Female=1) $\vee$ (Parents' advice=4 $\wedge$ Workaholism=3) $\vee$ (Age>24.0)
Techno, Trance	(Female=0) $\vee$ (Energy levels=5) $\vee$ (Workaholism=3 $\wedge$ Questionnaires or polls=2) $\vee$ (Healthy eating=4 $\wedge$ Getting angry=3) $\vee$ (Alcohol=2 $\wedge$ Personality=4)
Opera	(Charity=4 $\wedge$ Healthy eating=4) $\vee$ (Assertiveness=5 $\wedge$ Elections=5 $\wedge$ Funniness=5)

Now we look at the results from *GreedyRuleList*. Its conditions are concatenated with logical **OR**'s. Instance is evaluated by going through the conditions until the first one is met or none are.

Because of this, we have positive class probability in front of each condition, as stopping at different conditions gives different information about positive class probability.

GENRE	RULES
Dance	Energy levels > 3.5 (63.22%) ∨ Small - big dogs <= 2.5 (57.89%) ∨ Weight > 102.5 (100.00%) ELSE (34.75%)
Folk	Unpopularity > 4.5 (29.81%) ∨ Village > 0.5 (25.45%) ∨ Age > 24.5 (30.00%) ∨ Weight > 110.0 (66.67%) ∨ Hypochondria > 2.5 (17.91%) ELSE (5.68%)
Country	Smoking <= 0.5 (28.41%) ∨ Number of siblings > 3.5 (41.67%) ∨ Lying_everytime it suits me > 0.5 (22.03%) ELSE (9.24%)
Classical music	Socializing <= 1.5 (77.14%) ∨ Cheating in school <= 1.5 (91.67%) ∨ Height > 190.5 (77.78%) ELSE (41.79%)
Musical	Female > 0.5 (51.14%) ∨ Fake > 3.5 (52.63%) ∨ Elections > 4.5 (31.88%) ELSE (12.50%)
Pop	Knowing the right people > 2.5 (74.68%) ∨ Responding to a serious letter <= 1.5 (100.00%) ∨ Small - big dogs <= 2.5 (71.43%) ∨ Giving > 3.5 (71.43%) ∨ Age > 24.5 (100.00%) ELSE (16.67%)
Rock	Alcohol > 0.5 (82.14%) ∨ Questionnaires or polls > 1.5 (69.77%) ∨ Height > 188.5 (100.00%) ELSE (0.00%)
Metal or Hardrock	Female <= 0.5 (39.15%) ∨ Life struggles <= 1.5 (64.29%) ∨ Weight > 75.5 (70.00%) ∨ Giving <= 2.5 (32.26%) ∨ Criminal damage > 3.5 (22.92%) ∨ Age <= 15.5 (100.00%) ∨ Loneliness > 3.5 (16.67%) ∨ Number of siblings > 2.5 (23.08%) ∨ Health <= 2.5 (12.50%) ∨ God <= 1.5 (16.67%) ELSE (0.00%)
Punk	Hypochondria > 3.5 (50.00%) ∨ Workaholism > 2.5 (34.40%) ∨ Village > 0.5 (29.63%) ELSE (12.04%)
Hiphop, Rap	Personality > 3.5 (61.93%) ∨ Cheating in school > 3.5 (48.96%) ∨ Mood swings <= 1.5 (100.00%) ELSE (26.39%)
Reggae, Ska	Smoking > 0.5 (44.14%) ∨ Children <= 1.5 (100.00%) ∨ God <= 2.5 (41.67%) ∨ Finding lost valuables <= 2.5 (33.33%) ∨ Getting angry <= 1.5 (40.00%) ∨ Height > 190.5 (50.00%) ∨ Getting angry > 4.5 (33.33%) ELSE (0.00%)
Swing, Jazz	Workaholism > 2.5 (46.46%) ∨ Loneliness > 3.5 (52.63%) ∨ Decision making > 3.5 (43.59%) ∨ Changing the past > 4.5 (66.67%) ∨ Weight > 76.5 (32.00%) ∨ Hypochondria > 4.5 (100.00%) ∨ Age > 22.5 (33.33%) ∨ Number of siblings > 2.5 (33.33%) ELSE (0.00%)

Rock n roll	Alcohol > 0.5 (58.29%) ∨ Happiness in life > 4.5 (100.00%) ∨ Personality <= 2.5 (100.00%) ∨ Smoking <= 0.5 (38.89%) ∨ New environment <= 2.5 (100.00%) ∨ Daily events <= 1.5 (100.00%) ELSE (0.00%)
Alternative	Elections > 3.5 (51.35%) ∨ Achievements > 3.5 (49.12%) ∨ Waiting > 3.5 (56.52%) ∨ Hypochondria > 3.5 (50.00%) ∨ Parents' advice <= 2.5 (44.44%) ELSE (14.95%)
Latino	Female > 0.5 (56.46%) ∨ Education > 4.5 (100.00%) ∨ Parents' advice > 3.5 (29.69%) ELSE (12.98%)
Techno, Trance	Height > 177.5 (40.24%) ∨ Dreams > 4.5 (54.55%) ∨ Cheating in school > 2.5 (22.85%) ELSE (9.46%)
Opera	Charity > 2.5 (27.03%) ∨ Reliability > 3.5 (19.13%) ∨ Workaholism > 3.5 (12.50%) ELSE (0.00%)

Limitations and future work

Treating our 19 genres as independent targets divides the predictive signal. For future work, grouping these genres into psychological categories (e.g., clustering *Opera* and *Classical* into 'Sophisticated') could lead to better predictions of musical taste based on personality (Rentfrow et al., 2011). Some of the limitations of our paper are the following:

- **Sample size constraint:** The limited size of the dataset (N = 1010) restricts the overall results, reducing statistical power and limiting the complexity of patterns.
- **Low signal:** The predictive signal of personality traits, demographic traits and health habits against genre preferences is weak (*Schäfer & Mehlhorn, 2017*), which bounds model accuracy and rule coverage despite our model selection.
- **Demographic selection bias:** The survey is restricted to participants aged 15-30. Because of this, we can't generalize our findings to a broader age population

Conclusion

In this paper, we have presented a multi-labeling model for the prediction of the most promising musical genres a person might like based on health habits, personality traits, and demographic traits using multiple classification models. By employing a binary relevance approach, our final calibrated model successfully outperformed baseline approaches across key multi-label metrics. Additionally, we were able to extract interpretable rule sets using *Decision Trees*, *RIPPER* algorithm, and the *GreedyRuleList* algorithm, explicitly mapping specific user traits to genre preferences. Finally, while the relatively small margin of improvement confirms that the

predictive signal between individual differences and musical taste is inherently weak, our methodology provides a solid foundation for future work, such as grouping genres into broader psychological categories to improve prediction accuracy.

References

- Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. *Proceedings of the 22nd international conference on Machine learning - ICML '05*, 625–632.
- Rentfrow, P. J., Goldberg, L. R., & Levitin, D. J. (2011). The structure of musical preferences: A five-factor model. *Journal of Personality and Social Psychology*, 100, 1139-1157.
- Schäfer, T., & Mehlhorn, C. (2017). Can personality traits explain musical preferences? A meta-analysis on the correlations between musical style preferences and personality traits. *Personality and Individual Differences*, 116, 265–273.
- Szymański, P., & Kajdanowicz, T. (2017). A Network Perspective on Stratification of Multi-Label Data. *arXiv*.